Del testing manual a la automatización inteligente

Estrategia de automatización para PedidoHub

Luis Andrés Suárez Andrade · Aseguramiento de la Calidad del Software (SQA)
Universidad Privada del Valle · Lectura base: Crispin y Gregory, Agile Testing, Parte IV (caps. 13 y 14)

El problema: regresión manual que crece en cada entrega

- Sistema que cada dos semanas suma funciones y se prueba a mano antes de entregar.
- PedidoHub tiene la misma forma: login, pedidos, facturación, reportes y entregas incrementales.
- Pregunta de la dinámica: ¿qué automatizar, en qué nivel, con qué herramienta y cuándo ejecutarlo?
- Resultado: 216 pruebas automatizadas, 100 % en verde, 81.3 % de cobertura de líneas.

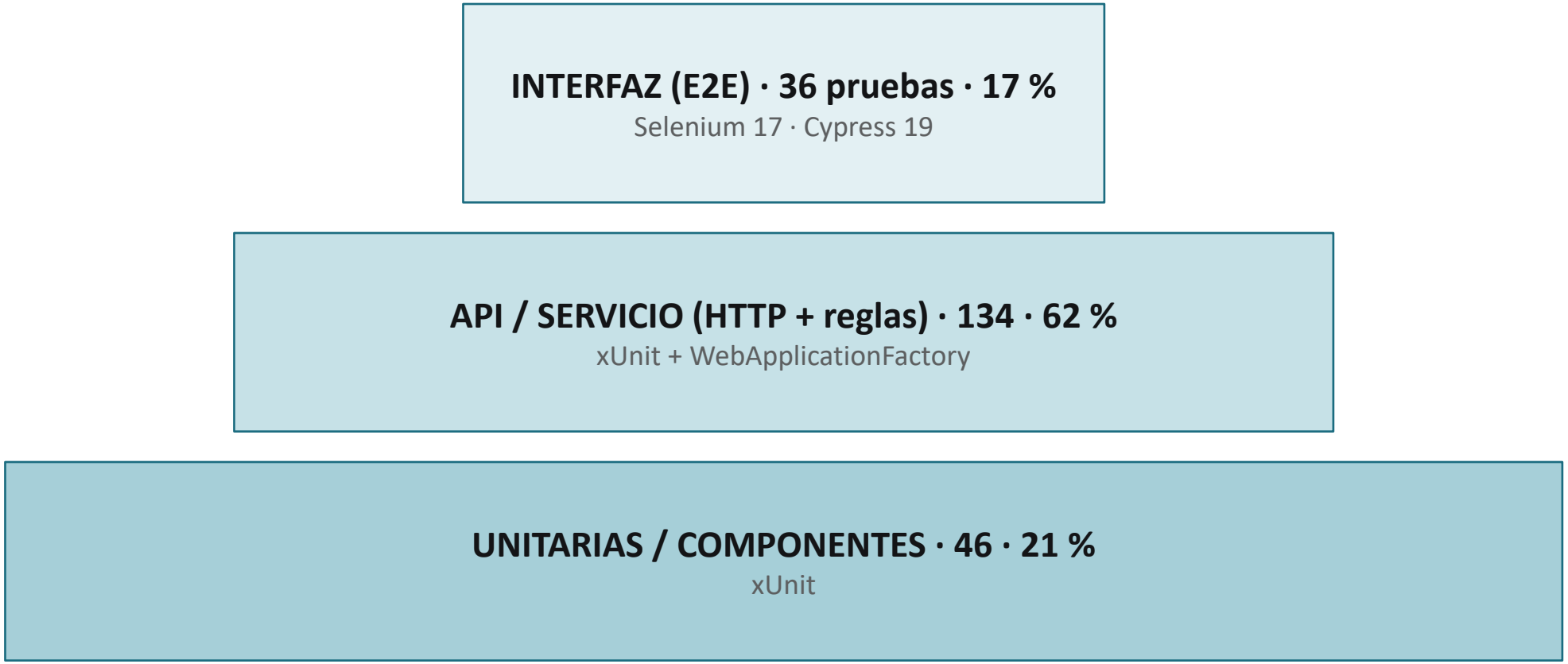
Fase 1 · Tres beneficios

Beneficio	Evidencia en PedidoHub
Retroalimentación rápida y defectos tempranos	180 pruebas en ~13 s; 5 defectos hallados con la prueba escrita antes del código
Libera tiempo para exploratorias	La regresión de 17 historias es automática; lo manual queda para usabilidad y plano de mesas
Red de seguridad y documentación ejecutable	Agregar login, roles y bitácora no rompió nada sin avisar; nombres Dado/Cuando/Entonces

Fase 1 · Tres obstáculos

Obstáculo	Evidencia en PedidoHub
Inversión inicial y curva de aprendizaje	SQLite no suma decimales; Cypress falló por un BOM. Se invirtió una vez y luego cada prueba es barata
Pruebas frágiles y mantenimiento en la interfaz	La nueva landing y el login obligaron a ajustar las E2E; un selector mal escrito dio un falso rojo
Entornos distintos de producción	Integración con SQLite vs producción con SQL Server; se mitiga con E2E contra SQL Server en CI

Fase 2 · La pirámide de PedidoHub



La capa media es la mayor porque es una aplicación MVC sin API JSON: las reglas viven en los controladores y se prueban por HTTP, sin navegador.

Fase 2 · Ejemplos y justificación por nivel

Nivel	Ejemplos reales	Por qué ahí
Unitarias	Total del pedido = Σ cantidad × precio · NIT de 5 a 20 dígitos · ranking de platos	Rápidas, sin BD ni navegador; señalan exactamente qué falló
API / servicio	Mesa con pedido abierto rechaza otro · mesero denegado en Empleados · CSV de ventas	Ejercitan reglas, seguridad y datos juntos y siguen siendo rápidas
Interfaz	Reservar 5 personas en mesa de 4 · login y acceso denegado en el navegador	Comprueban la integración real; lentas y frágiles, por eso pocas

Fase 3 · Matriz de decisiones: las tres primeras

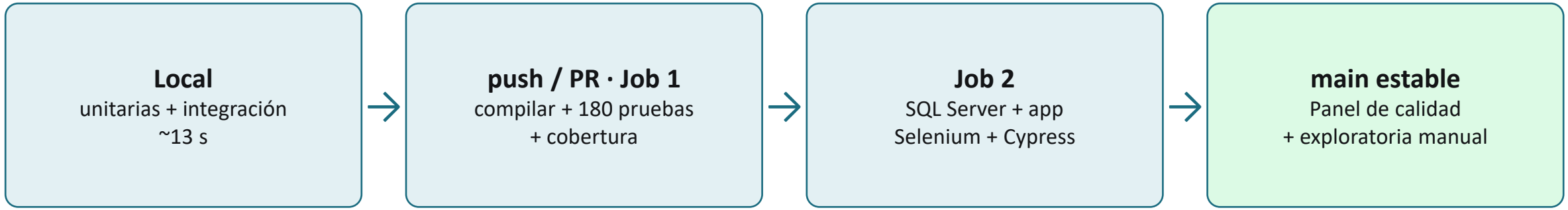
Prueba	Nivel · herramienta	Resultado esperado	Por qué primero
Cálculo de totales de pedido y factura	Unitaria · xUnit	Pedido 1 = 130.00; sin platos = 0; promedio = 150.75	Es dinero; se repite en cada cambio
Reglas de facturación y pedidos	Integración HTTP · xUnit + SQLite	Rechazo con mensaje exacto y BD sin cambios	Donde aparecieron los defectos
Login y permisos por rol	Integración + humo en Selenium y Cypress	Sin sesión → login; mesero → Acceso denegado	Puerta de entrada de todo el sistema

Manual: exploratorias del plano de mesas, usabilidad, aceptación con el usuario, redacción de mensajes, carga.

Fase 3 · Cómo evitar pruebas difíciles de mantener

- Cada regla en el nivel más bajo posible, sin duplicar en la interfaz
- Código de apoyo compartido (un único método de login por herramienta)
- Pruebas independientes: base en memoria por prueba y datos únicos
- Selectores estables por name y esperas explícitas
- Nombres Dado / Cuando / Entonces
- Una prueba roja se corrige el mismo día; nada se desactiva sin tarjeta
- Se depuran pruebas obsoletas en cada retrospectiva

Fase 4 · Integración continua: cuándo ejecutar cada prueba



Si una prueba falla:

- Detenerse: sin merge con el pipeline en rojo
- Clasificar en 15 min: defecto real, prueba frágil o entorno
- Corregir el mismo día; no desactivar para "que pase"
- Agregar prueba de regresión y revisarlo en la retrospectiva

Fase 5 · Defensa: ¿vale la pena el esfuerzo?

Métrica	Resultado
Tasa de éxito	216 / 216 = 100 %
Cobertura de líneas	81.3 % (meta 80 %)
Defectos encontrados y corregidos	6 / 6, cada uno con prueba de regresión
Tiempo de retroalimentación	13 s (unitarias + integración)
Densidad de defectos	3.7 por KLOC

Cierre

- Automatizar no es escribir scripts: es decidir qué, en qué nivel y cuándo.
- Primero automatizaría el cálculo de totales y las reglas de facturación.
- Eliminar todo lo manual sería el error: se perdería la exploración y la usabilidad.
- Demostración: /Calidad en PedidoHub y repositorio en GitHub (LuisSuarez11/PedidoHub_SQA).